

LC-3 Course Project Documentation: Part 1

Team Sign-Up & Task Assignment

- **Project Option Selected:** Option A: Bubble Sort
- **Team Members & Responsibilities:**
 - **[Hector Solis]:** Project Manager & Documentation Lead (Compiles final report, coordinates team submissions, and writes project purpose/outcome).
 - **[Rayan Khalid]:** Logic Architect & Pseudocode Designer (Designs core sorting logic, develops nested loop tracking, and determines conditional branching steps).
 - **[Ariyan Ghorbani]:** Subroutine & Systems Designer (Builds memory allocation plan, designs input/output conversion structures, and manages stack PUSH/POP parameters).

1. Project Purpose, Objectives, and Expected Outcome

Project Purpose

The purpose of this project is to apply core computer architecture and assembly programming principles by designing a functional LC-3 program. Specifically, this project implements a low-level Bubble Sort algorithm to organize user data. The assignment requires our team to manage direct hardware constraints, manually navigate memory spaces using pointers, control system execution via software stacks, and handle basic human-machine input/output interfacing.

Expected Outcome

The final outcome will be a fully functional, highly documented LC-3 assembly program. The program will prompt a user to input 8 individual integers between 0 and 100 via the console. It will cleanly store these values in a sequential memory array, execute a stable Bubble Sort algorithm to rearrange the dataset in ascending order, and output the correctly sorted integers back to the console screen.

Program Objectives (Chronological Order)

1. **Initialize System Space:** Establish the global hardware layout by defining the program origination address (.ORIG x3000), allocating a block of 8 words for storage (.BLKW #8), and initializing the hardware Stack Pointer register (R6 <- x4000).
2. **Execute Input Subroutine:** Invoke a dedicated subroutine using a subroutine call (JSR). This routine will use a system call directive (TRAP x23) to capture 8 user keystrokes from

- the console.
3. **Perform ASCII-to-Integer Conversion:** Inside the input routine, process each incoming character by subtracting x30 to translate the console text representation into a raw numeric integer value.
 4. **Populate Memory Array Using Pointers:** Store each converted integer into sequential memory addresses by incrementing a dedicated pointer register (STR and ADD pointer operations).
 5. **Protect Register Integrity (Save/Restore):** Prior to executing the sorting algorithm, use the stack pointer to execute PUSH operations, saving critical register states to memory so data is not corrupted during calculations.
 6. **Execute Bubble Sort Core Logic:** Implement nested loops utilizing iterative branching. The outer loop handles the passes (N-1), while the inner loop manages sequential traversals.
 7. **Implement Arithmetic Comparisons:** Load adjacent elements via relative pointers and subtract the second element from the first (utilizing Two's Complement addition: NOT then ADD #1).
 8. **Execute Conditional Branching & Data Swapping:** Evaluate the CPU condition codes (N, Z, P). If the first element is larger than the second (BRp), perform a safe memory swap; otherwise, bypass the swap (BRnz) and advance the pointer.
 9. **Restore Register Integrity:** Pop saved states off the stack right before exiting the sorting loop to guarantee standard data behavior.
 10. **Execute Output Subroutine:** Iterate exactly 8 times through the finalized memory array via a pointer to read out sorted numbers.
 11. **Perform Integer-to-ASCII Conversion:** Add x30 to each sorted numerical integer to transform it back into a readable console text character.
 12. **Display Sorted Array:** Output the elements to the monitor using system call directives (TRAP x21).
 13. **Terminate Program:** Securely halt the LC-3 processing unit using the final system command (TRAP x25).

2. Program Pseudocode

```

START
    // Global Setup
    SP <- x4000                // R6 acts as Stack Pointer
    ARRAY_BASE <- x3100        // Base address for data array

    // Program Flow Execution
    CALL SUB_GET_INPUT
    CALL SUB_BUBBLE_SORT
    CALL SUB_PRINT_OUTPUT
    HALT                      // TRAP x25

// =====
// SUBROUTINE: SUB_GET_INPUT
// =====

```

```

SUB_GET_INPUT:
    PUSH R1, R2, R0                // Save registers to stack
    R1 <- 8                        // Set loop counter to 8
    R2 <- ARRAY_BASE               // Set pointer to array storage

INPUT_LOOP:
    GETC                          // TRAP x23: Read character from console
    R0 <- R0 - x30                 // ASCII Conversion: Char to raw integer
    STR R0, R2, #0                // Store integer at current pointer
location
    R2 <- R2 + 1                  // Increment memory pointer
    R1 <- R1 - 1                  // Decrement counter
    BRp INPUT_LOOP               // Continue until 8 numbers are stored

    POP R0, R2, R1               // Restore original register states
    RET                          // Return to main line

// =====
// SUBROUTINE: SUB_BUBBLE_SORT
// =====
SUB_BUBBLE_SORT:
    PUSH R1, R2, R3, R4, R5, R0 // Save registers to stack
    R1 <- 7                      // Outer loop pass counter (N-1 = 7)

OUTER_LOOP:
    BRz SORT_DONE               // If passes remaining is 0, exit sort
    R2 <- R1                    // Inner loop counter matches current
pass depth
    R3 <- ARRAY_BASE            // Reset tracking pointer to start of
array

INNER_LOOP:
    LDR R4, R3, #0              // Load Element 1
    LDR R5, R3, #1              // Load Element 2 (Adjacent)

    // Math comparison: Compute R4 - R5
    NOT R0, R5
    ADD R0, R0, #1              // Two's complement of Element 2 (-R5)
    ADD R0, R4, R0              // R0 = Element 1 - Element 2

    BRnz NO_SWAP                // If Element 1 <= Element 2, no action
needed

    // Swap Execution
    STR R5, R3, #0              // Write Element 2 into Element 1's
position
    STR R4, R3, #1              // Write Element 1 into Element 2's
position

```

```

NO_SWAP:
    R3 <- R3 + 1          // Move pointer forward to next pair
    R2 <- R2 - 1          // Decrement inner loop counter
    BRp INNER_LOOP       // Repeat inner comparison pass

    R1 <- R1 - 1          // Decrement outer pass counter
    BRg OUTER_LOOP       // Repeat outer sorting pass

SORT_DONE:
    POP R0, R5, R4, R3, R2, R1 // Restore original register states
    RET                    // Return to main line

// =====
// SUBROUTINE: SUB_PRINT_OUTPUT
// =====
SUB_PRINT_OUTPUT:
    PUSH R1, R2, R0        // Save registers to stack
    R1 <- 8                // Set loop counter to 8
    R2 <- ARRAY_BASE       // Point to start of sorted data

OUTPUT_LOOP:
    LDR R0, R2, #0         // Load integer from current pointer
    R0 <- R0 + x30         // ASCII Conversion: Integer back to text
char
    OUT                    // TRAP x21: Write character to console
    R2 <- R2 + 1           // Advance pointer
    R1 <- R1 - 1           // Decrement loop counter
    BRp OUTPUT_LOOP       // Continue until all 8 elements are
displayed

    POP R0, R2, R1        // Restore original register states
    RET                    // Return to main line

```

3. Testing and Target Dataset Verification Plan

To meet the evaluation criteria outlined under Criterion 13, the logic has been structured to evaluate the following static data block:

- **Target Inputs:** 11, 8, 2, 17, 6, 4, 3, 21
- **Target Outputs:** 2, 3, 4, 6, 8, 11, 17, 21

The algorithm handles this by iteratively bubbling the highest numerical integers to the end of the x3100 block during each pass of the nested loops, resulting in a strictly sorted ascending dataset.